

第2章 运算符、表达式与内置对象

2.1 Python常用内置对象

- 对象是python语言中最基本的概念，在python中处理的一切都是对象。
- python中有许多内置对象可供编程者使用，内置对象可直接使用，如数字、字符串、列表、del等。
- 非内置对象需要导入模块才能使用，如正弦函数 $\sin(x)$ ，随机数产生函数`random()`等。

2.1 Python常用内置对象

常用内置对象

对象类型	类型名称	示例	简要说明
数字	int float complex	1234 3.14, 1.3e5 3+4j	数字大小没有限制，内置支持复数及其运算
字符串	str	'swfu', "I'm student", "Python ", r'abc', R'bcd'	使用单引号、双引号、三引号作为定界符，以字母r或R引导的表示原始字符串
字节串	bytes	b'hello world'	以字母b引导，可以使用单引号、双引号、三引号作为定界符
列表	list	[1, 2, 3] ['a', 'b', ['c', 2]]	所有元素放在一对方括号中，元素之间使用逗号分隔，其中的元素可以是任意类型
字典	dict	{1:'food', 2:'taste', 3:'import'}	所有元素放在一对大括号中，元素之间使用逗号分隔， 元素形式为“键:值”
元组	tuple	(2, -5, 6) (3,)	不可变 ，所有元素放在一对圆括号中，元素之间使用逗号分隔， 如果元组中只有一个元素的话，后面的逗号不能省略
集合	set frozenset	{ 'a', 'b', 'c' }	所有元素放在一对大括号中，元素之间使用逗号分隔， 元素不允许重复 ；另外，set是可变的，而frozenset是不可变的

2.1 Python常用内置对象

续表

对象类型	类型名称	示例	简要说明
布尔型	bool	True, False	逻辑值，关系运算符、成员测试运算符、同一性测试运算符组成的表达式的值一般为True或False
空类型	NoneType	None	空值
异常	Exception ValueError TypeError		Python内置大量异常类，分别对应不同类型的异常
文件		f = open('data.dat', 'rb')	open是Python内置函数，使用指定的模式打开文件，返回文件对象
其他可迭代对象		生成器对象、range对象、zip对象、enumerate对象、map对象、filter对象等等	具有 惰性求值 的特点，除range对象之外，其他对象中的元素只能看一次
编程单元		函数（使用def定义） 类（使用class定义） 模块（类型为module）	类和函数都属于 可调用对象 ，模块用来集中存放函数、类、常量或其他对象

2.1.1 常量与变量

- 在Python中，**不需要事先声明变量名及其类型**，直接赋值即可创建各种类型的对象变量。这一点适用于Python任意类型的对象。

例如语句

```
>>> x = 3
```

凭空出现一个整型变量x

创建了整型变量x，并赋值为3，再例如语句

```
>>> x = 'Hello world.'
```

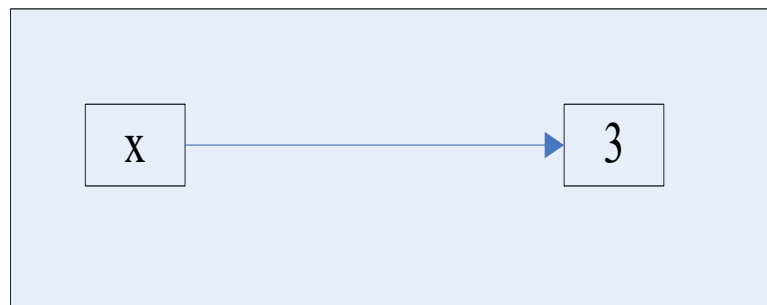
新的字符串变量，再也不是原来的x了

创建了字符串变量x，并赋值为'Hello world.'。

2.1.1 常量与变量

- 赋值语句的执行过程是：首先把等号右侧表达式的值计算出来，然后在内存中寻找一个位置把值存放进去，最后创建变量并指向这个内存地址。

```
>>> x = 3
```



- Python中的变量并不直接存储值，而是存储了值的内存地址或者引用，这也是变量类型随时可以改变的原因。

2.1.1 常量与变量

❖ Python属于**强类型编程语言**，Python解释器会根据赋值或运算来自动**推断**变量类型。Python还是一种**动态类型语言**，变量的类型也是可以随时变化的。

```
>>> x = 3
>>> print(type(x))
<class 'int'>
>>> x = 'Hello world.'
>>> print(type(x))          #查看变量类型
<class 'str'>
>>> x = [1,2,3]
>>> print(type(x))
<class 'list'>
>>> isinstance(3, int)      #测试对象是否是某个类型的实例
True
>>> isinstance('Hello world', str)
True
```

2.1.1 常量与变量

- 在定义变量名的时候，需要注意以下问题：
 - ✓ 变量名**必须**以字母或下划线开头，但以下划线开头的变量在Python中有特殊含义；
 - ✓ 变量名中**不能**有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等等）；
 - ✓ **不能**使用关键字作变量名，可以导入keyword模块后使用print(keyword.kwlist)查看所有Python关键字；
 - ✓ 变量名对英文字母的**大小写敏感**，例如student和Student是不同的变量。（使用时强制改为大小写）
x= 'AB' y=x.lower(), print(x) print(y) .upper() .lower()
 - ✓ **不建议**使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义。

2.1.2 数字

- 在Python中，内置的数字类型有整数、实数和复数。
- 整数类型除了常见的十进制整数，还有：
 - ✓ **二进制**。以0b开头，每一位只能是0或1。（**Binary**）
 - ✓ **八进制**。以0o开头，每一位只能是0、1、2、3、4、5、6、7这八个数字之一。（**Octal**）
 - ✓ **十六进制**。以0x开头，每一位只能是0、1、2、3、4、5、6、7、8、9、a、b、c、d、e、f之一。（**hexadecimal**）
- ✓ 二进制 bit 信息量

2.1.2 数字

- Python支持任意大的数字，具体可以大到什么程度仅受内存大小的限制。
- 由于精度的问题，对于实数运算可能会有一定的误差，应尽量避免在实数之间直接进行相等性测试，而是应该以二者之差的绝对值是否足够小作为两个实数是否相等的依据。
- 在数字的算术运算表达式求值时会进行隐式的类型转换，如果存在复数则都变成复数，如果没有复数但是有实数就都变成实数，如果都是整数则不进行类型转换（/运算符除外）。

2.1.2 数字

```
>>> 9999 ** 99                                     #这里**是幂乘运算符，等价于内置函数pow()
99014835352672348760226312475328262557055952889579105732432652912179483789405351346442
21768269164339325869243866777662440320016237568214004329750512088202049800987355527038
41362304669970510691243800218202840374329378800694920309791954185117798434329591212159
10629869993866990806757337472433120894242554489391091007320504903165678922088956073296
2926226305865706593594917896276756396848514900989999
>>> 0.3 + 0.2                                     #实数相加
0.5
>>> 0.4 - 0.1                                     #实数相减，结果稍微有点偏差
0.30000000000000004
>>> 0.4 - 0.1 == 0.3                             #应尽量避免直接比较两个实数是否相等
False
>>> abs(0.4-0.1 - 0.3) < 1e-6                   #这里1e-6表示10的-6次方
True

浮点数 0.1+0.2 0.3
0.3 → 0.3000000000000000444089209850062616169452667236328125
二进制表示浮点数时存在截断误差（转二再转十产生误差），一般从设计程序角度出发，判断条件为比值大小
true false
```

2.1.2 数字

- Python内置支持复数类型及其运算，并且形式与数学上的复数完全一致。

```
>>> x = 3 + 4j          #使用j或J表示复数虚部
>>> y = 5 + 6j
>>> x + y               #支持复数之间的加、减、乘、除以及幂乘等运算
(8+10j)
>>> x * y
(-9+38j)
>>> abs(x)              #内置函数abs()可用来计算复数的模   Absolute value
5.0
>>> x.imag              #虚部      Imaginary
4.0
>>> x.real              #实部      Real number
3.0
>>> x.conjugate()       #共轭复数   conjugate
(3-4j)
```

2.1.2 数字

- Python 3.6.x支持在数字中间位置使用单个下划线作为分隔来提高数字的可读性，类似于数学上使用逗号作为千位分隔符。
- 在Python数字中单个下划线可以出现在中间任意位置，但不能出现开头和结尾位置，也不能使用多个连续的下划线。

```
>>> 1_000_000
```

```
1000000
```

```
>>> 1_2_3_4
```

```
1234
```

```
>>> 1_2 + 3_4j
```

```
(12+34j)
```

```
>>> 1_2.3_45
```

```
12.345
```

2.1.3 字符串

- 在Python中，没有字符常量和变量的概念，只有字符串类型的常量和变量，**单个字符也是字符串**。使用**单引号、双引号、三单引号、三双引号**作为定界符（delimiter）来表示字符串，并且**不同的定界符之间可以互相嵌套**。
- Python 3.x全面支持中文，中文和英文字母都作为一个字符对待，甚至**可以使用中文作为变量名**。
- 除了支持使用加号运算符连接字符串以外，Python字符串还提供了大量的方法支持格式化、查找、替换、排版等操作。

2.1.3 字符串

```
>>> x = 'Hello world.'
>>> x = "Python is a great language."
>>> x = '''Tom said, "Let's go."'''
>>> print(x)
Tom said, "Let's go."
>>> x = 'good ' + 'morning'
>>> x
'good morning'
>>> x = 'good ' 'morning'
>>> x
'good morning'
>>> x = 'good '
>>> x = x'morning'
SyntaxError: invalid syntax
>>> x = x + 'morning'
>>> x
'good morning'
```

#使用单引号作为定界符
#使用双引号作为定界符
#不同定界符之间可以互相嵌套

#连接字符串

#连接字符串, 仅适用于字符串常量

#不适用于字符串变量

#字符串变量之间的连接可以使用加号

2.1.4 列表、元组、字典、集合

	列表	元组	字典	集合
类型名称	<code>list</code>	<code>tuple</code>	<code>dict</code>	<code>set</code>
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	是（使用序号作为下标）	是（使用序号作为下标）	是（使用“键”作为下标）	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希 （不可变的数据结构， <code>str tuple</code> ）
对元素值的要求	无	无	“键”必须可哈希	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	非常慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快 其他位置慢	不允许	快	快

2.1.4 列表、元组、字典、集合

```
>>> x_list = [1, 2, 3]
>>> x_tuple = (1, 2, 3)
>>> x_dict = {'a':97, 'b':98, 'c':99}
>>> x_set = {1, 2, 3}
>>> print(x_list[1])
2
>>> print(x_tuple[1])
2
>>> print(x_dict['a'])
97
>>> 3 in x_set
True
```

#创建列表对象
#创建元组对象
#创建字典对象
#创建集合对象
#使用下标访问指定位置的元素

#元组也支持使用序号作为下标

#字典对象的下标是“键”

#成员测试

2.2 Python运算符与表达式

- 在Python中，**单个常量或变量可以看作最简单的表达式**，使用除赋值运算符之外的其他任意运算符和函数调用连接的式子也属于表达式。

2.2 Python运算符与表达式

- 运算符优先级遵循的规则为：算术运算符优先级最高，其次是位运算符、成员测试运算符、关系运算符、逻辑运算符等，算术运算符遵循“先乘除，后加减”的基本运算原则。
- 虽然Python运算符有一套严格的优先级规则，但是强烈建议在编写复杂表达式时使用圆括号来明确说明其中的逻辑来提高代码可读性。

2.2 Python运算符与表达式

运算符	功能说明
+	算术加法，列表、元组、字符串合并与连接，正号
-	算术减法，集合差集，相反数
*	算术乘法，序列重复
/	真除法
//	求整商，但如果操作数中有实数的话，结果为实数形式的整数
%	求余数，字符串格式化
**	幂运算
<、<=、>、>=、==、!=	（值）大小比较，集合的包含关系比较
or	逻辑或
and	逻辑与
not	逻辑非
in	成员测试
is	对象同一性测试，即测试是否为同一个对象或内存地址是否相同
、^、&、<<、>>、~	位或、位异或、位与、左移位、右移位、位求反
&、 、^	集合交集、并集、对称差集
@	矩阵相乘运算符

2.2.1 算术运算符

(1) **+运算符**除了用于算术加法以外，还可以用于列表、元组、字符串的连接，但不支持不同类型的对象之间相加或连接。

```
>>> [1, 2, 3] + [4, 5, 6]           #连接两个列表
[1, 2, 3, 4, 5, 6]
>>> (1, 2, 3) + (4,)                #连接两个元组
(1, 2, 3, 4)
>>> 'abcd' + '1234'                 #连接两个字符串
'abcd1234'
>>> 'A' + 1                          #不支持字符与数字相加，抛出异常
TypeError: Can't convert 'int' object to str implicitly
>>> True + 3                          #Python内部把True当作1处理
4
>>> False + 3                        #把False当作0处理
3
```

2.2.1 算术运算符

(2) ***运算符**除了表示算术乘法，还可用于列表、元组、字符串这几个序列类型与整数的乘法，表示序列元素的重复，生成新的序列对象。**字典和集合不支持与整数的相乘**，因为其中的元素是不允许重复的。

```
>>> True * 3
```

```
3
```

```
>>> False * 3
```

```
0
```

```
>>> [1, 2, 3] * 3
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
>>> (1, 2, 3) * 3
```

```
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

```
>>> 'abc' * 3
```

```
'abcabcabc'
```

2.2.1 算术运算符

(3) **运算符/和//**在Python中分别表示算术除法和算术求整商 (floor division)

。

>>> 3 / 2 #数学意义上的除法

1.5

>>> 15 // 4 #如果两个操作数都是整数，结果为整数

3

>>> 15.0 // 4 #如果操作数中有实数，结果为实数形式的整数值

3.0

>>> -15//4 #向下取整

-4

2.2.1 算术运算符

(4) **%运算符**可以用于整数或实数的求余数运算，还可以用于字符串格式化，但是这种用法并不推荐。

```
>>> 789 % 23          #余数
```

```
7
```

```
>>> 123.45 % 3.2       #可以对实数进行余数运算，注意精度问题
```

```
1.8499999999999996
```

```
>>> '%c, %d' % (65, 65)  #把65分别格式化为字符和整数
```

```
'A, 65'
```

```
>>> '%f,%s' % (65, 65)   #把65分别格式化为实数和字符串
```

```
'65.000000,65'
```

2.2.1 算术运算符

(5) ****运算符**表示幂乘:

```
>>> 3 ** 2
```

#3的2次方, 等价于pow(3, 2)

```
9
```

```
>>> pow(3, 2, 8)
```

#等价于(3**2) % 8

```
1
```

```
>>> 9 ** 0.5
```

#9的0.5次方, 平方根

```
3.0
```

```
>>> (-9) ** 0.5
```

#可以计算负数的平方根

```
(1.8369701987210297e-16+3j)
```

2.2.2 关系运算符

- Python关系运算符最大的特点是**可以连用**。使用关系运算符的一个最重要的前提是，**操作数之间必须可比较大小**。

```
>>> 1 < 3 < 5                                #等价于1 < 3 and 3 < 5
```

```
True
```

```
>>> 3 < 5 > 2
```

```
True
```

```
>>> 1 > 6 < 8
```

```
False
```

```
>>> 1 > 6 < math.sqrt(9)                      #具有惰性求值或者逻辑短路的特点
```

```
False
```

```
>>> 1 < 6 < math.sqrt(9)                      #还没有导入math模块，抛出异常
```

```
NameError: name 'math' is not defined
```

```
>>> import math
```

```
>>> 1 < 6 < math.sqrt(9)
```

```
False
```

2.2.2 关系运算符

>>> 'Hello' > 'world'	#比较字符串大小
False	
>>> [1, 2, 3] < [1, 2, 4]	#比较列表大小
True	
>>> 'Hello' > 3	#字符串和数字不能比较
TypeError: unorderable types: str() > int()	
>>> {1, 2, 3} < {1, 2, 3, 4}	#测试是否子集
True	
>>> {1, 2, 3} == {3, 2, 1}	#测试两个集合是否相等
True	
>>> {1, 2, 4} > {1, 2, 3}	#集合之间的包含测试
False	
>>> {1, 2, 4} < {1, 2, 3}	
False	
>>> {1, 2, 4} == {1, 2, 3}	
False	

2.2.3 成员测试运算符in

- 成员测试运算符in用于成员测试，即测试一个对象是否为另一个对象的元素。

```
>>> 3 in [1, 2, 3]
```

```
True
```

```
>>> 5 in range(1, 10, 1)
```

```
True
```

```
>>> 'abc' in 'abcdefg'
```

```
True
```

```
>>> for i in (3, 5, 7):  
    print(i, end='\t')
```

```
3      5      7
```

#测试3是否存在于列表[1, 2, 3]中

#range()是用来生成指定范围数字的内置函数

#子字符串测试

#循环，成员遍历

2.2.4 集合运算符

- 集合的交集、并集、对称差集等运算借助于 $\&$ 、 $|$ 、 $\wedge$ 来实现，而差集则使用减号运算符实现（注意，并集运算符不是加号）。

<pre>>>> {1, 2, 3} {3, 4, 5} {1, 2, 3, 4, 5}</pre>	#并集，自动去除重复元素
<pre>>>> {1, 2, 3} & {3, 4, 5} {3}</pre>	#交集
<pre>>>> {1, 2, 3} ^ {3, 4, 5} {1, 2, 4, 5}</pre>	#对称差集
<pre>>>> {1, 2, 3} - {3, 4, 5} {1, 2}</pre>	#差集

2.2.5 逻辑运算符

- 逻辑运算符and、or、not常用来连接条件表达式构成更加复杂的条件表达式，并且and和or具有惰性求值或逻辑短路的特点，当连接多个表达式时只计算必须要计算的值。
- 另外要注意的是，运算符and和or并不一定会返回True或False，而是得到最后一个被计算的表达式的值，但是运算符not一定会返回True或False。

2.2.5 逻辑运算符

```
>>> 3>5 and a>3
```

```
False
```

```
>>> 3>5 or a>3
```

```
NameError: name 'a' is not defined
```

```
>>> 3<5 or a>3
```

```
True
```

```
>>> 3 and 5
```

```
5
```

```
>>> 3 and 5>2
```

```
True
```

```
>>> 3 not in [1, 2, 3]
```

```
False
```

```
>>> 3 is not 5
```

```
True
```

```
>>> not 3
```

```
False
```

```
>>> not 0
```

```
True
```

#注意, 此时并没有定义变量a

#3>5的值为False, 所以需要计算后面表达式

#3<5的值为True, 不需要计算后面表达式

#最后一个计算的表达式的值作为整个表达式的值

#逻辑非运算not

#not的计算结果只能是True或False之一

2.2.6 补充说明

- Python还有赋值运算符=和+=、-=、*=、/=、//=、**=、|=、^=等大量复合赋值运算符。
- Python不支持++和--运算符，虽然在形式上有时候似乎可以这样用，但实际上是另外的含义，要注意和其他语言的区别。

2.2.6 补充说明

```
>>> i = 3
>>> ++i
3
>>> +(3)
3
>>> i++
SyntaxError: invalid syntax
>>> --i
3
>>> ---i
-3
>>> i--
SyntaxError: invalid syntax
```

#正正得正

#与++i等价

#Python不支持++运算符, 语法错误

#负负得正, 等价于-(-i)

#等价于-(-(-i))

#Python不支持--运算符, 语法错误

2.3 Python常用内置函数用法精要

- 内置函数（BIF, built-in functions）是Python内置对象类型之一，**不需要额外导入任何模块即可直接使用**，这些内置对象都封装在内置模块__builtins__之中，用C语言实现并且进行了大量优化，具有非常快的运行速度，**推荐优先使用**。使用内置函数dir()可以查看所有内置函数和内置对象：

```
>>> dir(__builtins__)
```

- 使用help(函数名)可以查看某个函数的用法。

```
>>> help(sum)
```

```
Help on built-in function sum in module builtins:
```

```
sum(iterable, start=0, /)
```

```
Return the sum of a 'start' value (default: 0) plus an iterable of numbers
```

```
When the iterable is empty, return the start value.
```

```
This function is intended specifically for use with numeric values and may reject non-numeric types.
```

2.3 Python常用内置函数用法精要

函数	功能简要说明
<code>abs(x)</code>	返回数字x的绝对值或复数x的模
<code>all(iterable)</code>	如果对于可迭代对象中所有元素x都等价于True，也就是对于所有元素x都有 <code>bool(x)</code> 等于True，则返回True。对于空的可迭代对象也返回True
<code>any(iterable)</code>	只要可迭代对象iterable中存在元素x使得 <code>bool(x)</code> 为True，则返回True。对于空的可迭代对象，返回False
<code>ascii(obj)</code>	把对象转换为ASCII码表示形式，必要的时候使用转义字符来表示特定的字符
<code>bin(x)</code>	把整数x转换为二进制串表示形式
<code>bool(x)</code>	返回与x等价的布尔值True或False
<code>bytes(x)</code>	生成字节串，或把指定对象x转换为字节串表示形式
<code>callable(obj)</code>	测试对象obj是否可调用。类和函数是可调用的，包含 <code>__call__()</code> 方法的类的对象也是可调用的
<code>compile()</code>	用于把Python代码编译成可被 <code>exec()</code> 或 <code>eval()</code> 函数执行的代码对象
<code>complex(real, [imag])</code>	返回复数
<code>chr(x)</code>	返回Unicode编码为x的字符

2.3 Python常用内置函数用法精要

续表1

函数	功能简要说明
<code>delattr(obj, name)</code>	删除属性，等价于 <code>del obj.name</code>
<code>dir(obj)</code>	返回指定对象或模块 <code>obj</code> 的成员列表，如果不带参数则返回当前作用域内所有标识符
<code>divmod(x, y)</code>	返回包含整商和余数的元组 $((x-x\%y)/y, x\%y)$
<code>enumerate(iterable[, start])</code>	返回包含元素形式为 $(0, \text{iterable}[0])$, $(1, \text{iterable}[1])$, $(2, \text{iterable}[2])$, ... 的迭代器对象
<code>eval(s[, globals[, locals]])</code>	计算并返回字符串 <code>s</code> 中表达式的值
<code>exec(x)</code>	执行代码或代码对象 <code>x</code>
<code>exit()</code>	退出当前解释器环境
<code>filter(func, seq)</code>	返回 <code>filter</code> 对象，其中包含序列 <code>seq</code> 中使得单参数函数 <code>func</code> 返回值为 <code>True</code> 的那些元素，如果函数 <code>func</code> 为 <code>None</code> 则返回包含 <code>seq</code> 中等于 <code>True</code> 的元素的 <code>filter</code> 对象
<code>float(x)</code>	把整数或字符串 <code>x</code> 转换为浮点数并返回
<code>frozenset([x])</code>	创建不可变的集合对象
<code>getattr(obj, name[, default])</code>	获取对象中指定属性的值，等价于 <code>obj.name</code> ，如果不存在指定属性则返回 <code>default</code> 的值，如果要访问的属性不存在并且没有指定 <code>default</code> 则抛出异常

2.3 Python常用内置函数用法精要

续表2

函数	功能简要说明
<code>globals()</code>	返回包含当前作用域内全局变量及其值的字典
<code>hasattr(obj, name)</code>	测试对象obj是否具有名为name的成员
<code>hash(x)</code>	返回对象x的哈希值，如果x不可哈希则抛出异常
<code>help(obj)</code>	返回对象obj的帮助信息
<code>hex(x)</code>	把整数x转换为十六进制串
<code>id(obj)</code>	返回对象obj的标识（内存地址）
<code>input([提示])</code>	显示提示，接收键盘输入的内容，返回字符串
<code>int(x[, d])</code>	返回实数（float）、分数（Fraction）或高精度实数（Decimal）x的整数部分，或把d进制的字符串x转换为十进制并返回，d默认为十进制
<code>isinstance(obj, class-or-type-or-tuple)</code>	测试对象obj是否属于指定类型（如果有多个类型的话需要放到元组中）的实例
<code>iter(...)</code>	返回指定对象的可迭代对象
<code>len(obj)</code>	返回对象obj包含的元素个数，适用于列表、元组、集合、字典、字符串以及range对象和其他可迭代对象

2.3 Python常用内置函数用法精要

续表3

函数	功能简要说明
<code>list([x])</code> 、 <code>set([x])</code> 、 <code>tuple([x])</code> 、 <code>dict([x])</code>	把对象x转换为列表、集合、元组或字典并返回，或生成空列表、空集合、空元组、空字典
<code>locals()</code>	返回包含当前作用域内局部变量及其值的字典
<code>map(func, *iterables)</code>	返回包含若干函数值的map对象，函数func的参数分别来自于iterables指定的每个迭代对象，
<code>max(x)</code> 、 <code>min(x)</code>	返回可迭代对象x中的最大值、最小值，要求x中的所有元素之间可比较大小，允许指定排序规则和x为空时返回的默认值
<code>next(iterator[, default])</code>	返回可迭代对象x中的下一个元素，允许指定迭代结束之后继续迭代时返回的默认值
<code>oct(x)</code>	把整数x转换为八进制串
<code>open(name[, mode])</code>	以指定模式mode打开文件name并返回文件对象
<code>ord(x)</code>	返回1个字符x的Unicode编码
<code>pow(x, y, z=None)</code>	返回x的y次方，等价于 <code>x ** y</code> 或 <code>(x ** y) % z</code>

2.3 Python常用内置函数用法精要

续表4

函数	功能简要说明
<code>print(value, ..., sep=' ', end=' \n', file=sys.stdout, flush=False)</code>	基本输出函数
<code>quit()</code>	退出当前解释器环境
<code>range([start,] end [, step])</code>	返回range对象，其中包含左闭右开区间[start,end)内以step为步长的整数
<code>reduce(func, sequence[, initial])</code>	将双参数的函数func以迭代的方式从左到右依次应用至序列seq中每个元素，最终返回单个值作为结果。在Python 2.x中该函数为内置函数，在Python 3.x中需要从functools中导入reduce函数再使用
<code>repr(obj)</code>	返回对象obj的规范化字符串表示形式，对于大多数对象有 <code>eval(repr(obj))==obj</code>
<code>reversed(seq)</code>	返回seq（可以是列表、元组、字符串、range以及其他可迭代对象）中所有元素逆序后的迭代器对象

2.3 Python常用内置函数用法精要

续表5

函数	功能简要说明
<code>round(x [, 小数位数])</code>	对x进行四舍五入，若不指定小数位数，则返回整数
<code>sorted(iterable, key=None, reverse=False)</code>	返回排序后的列表，其中iterable表示要排序的序列或迭代对象 key用来指定排序规则或依据，reverse用来指定升序或降序。该函数不改变iterable内任何元素的顺序
<code>str(obj)</code>	把对象obj直接转换为字符串
<code>sum(x, start=0)</code>	返回序列x中所有元素之和，返回start+sum(x)
<code>type(obj)</code>	返回对象obj的类型
<code>zip(seq1 [, seq2 [...]])</code>	返回zip对象，其中元素为(seq1[i], seq2[i], ...)形式的元组 最终结果中包含的元素个数取决于所有参数序列或可迭代对象中最短的那个

2.3.1 类型转换与类型判断

- 内置函数bin()、oct()、hex()用来将整数转换为二进制、八进制和十六进制形式，这三个函数都**要求参数必须为整数**。

```
>>> bin(555)                #把数字转换为二进制串
'0b1000101011'

>>> oct(555)                #转换为八进制串
'0o1053'

>>> hex(555)                #转换为十六进制串
'0x22b'
```

2.3.1 类型转换与类型判断

- 内置函数float()用来将其他类型数据转换为实数，complex()可以用来生成复数。

```
>>> float(3)
```

```
3.0
```

#把整数转换为实数

```
>>> float('3.5')
```

```
3.5
```

#把数字字符串转换为实数

```
>>> float('inf')
```

```
inf
```

#无穷大，其中inf不区分大小写

```
>>> complex(3)
```

```
(3+0j)
```

#指定实部

```
>>> complex(3, 5)
```

```
(3+5j)
```

#指定实部和虚部

```
>>> complex('inf')
```

```
(inf+0j)
```

#无穷大

2.3.1 类型转换与类型判断

- ord()和chr()是一对功能相反的函数，ord()用来返回单个字符的Unicode码，而chr()则用来返回Unicode编码对应的字符，str()则直接将其任意类型参数转换为字符串。

```
>>> ord('a')           #查看指定字符的Unicode编码
97
>>> chr(65)            #返回数字65对应的字符
'A'
>>> chr(ord('A')+1)     #Python不允许字符串和数字之间的加法操作
'B'
>>> chr(ord('国')+1)    #支持中文
'图'
>>> ord('董')           #这个用法仅适用于Python 3.x
33891
```

2.3.1 类型转换与类型判断

```
>>> str(1234)
```

```
'1234'
```

#直接变成字符串

```
>>> str([1,2,3])
```

```
'[1, 2, 3]'
```

```
>>> str((1,2,3))
```

```
'(1, 2, 3)'
```

```
>>> str({1,2,3})
```

```
'{1, 2, 3}'
```

2.3.1 类型转换与类型判断

- list()、tuple()、dict()、set()用来把其他类型的数据转换成为列表、元组、字典、集合，或者创建空列表、空元组、空字典和空集合。

```
>>> list(range(5))           #把range对象转换为列表
[0, 1, 2, 3, 4]
>>> tuple(_)                 #一个下划线表示上一次正确的输出结果
(0, 1, 2, 3, 4)
>>> dict(zip('1234', 'abcde')) #创建字典
{'1': 'a', '2': 'b', '3': 'c', '4': 'd'}
>>> set('1112234')           #创建可变集合，自动去除重复
{'1', '2', '3', '4'}
```

2.3.1 类型转换与类型判断

- 内置函数type()和isinstance()可以用来判断数据类型，常用来对函数参数进行检查，可以避免错误的参数类型导致函数崩溃或返回意料之外的结果。

```
>>> type(3)                                #查看3的类型
<class 'int'>
>>> type([3])                              #查看[3]的类型
<class 'list'>
>>> type({3}) in (list, tuple, dict)        #判断{3}是否为list,tuple或dict类型的实例
False
>>> type({3}) in (list, tuple, dict, set)   #判断{3}是否为list,tuple,dict或set的实例
True
>>> isinstance(3, int)                     #判断3是否为int类型的实例
True
>>> isinstance(3j, int)
False
>>> isinstance(3j, (int, float, complex))  #判断3是否为int,float或complex类型
True
```

2.3.1 类型转换与类型判断

- 内置函数`eval()` *evaluation*用来计算字符串的值，在有些场合也可以用来实现类型转换的功能。

```
>>> eval('3+5')
```

```
8
```

```
>>> eval('9')
```

#把数字字符串转换为数字

```
9
```

```
>>> eval('09')
```

#抛出异常，不允许以0开头的数字

```
SyntaxError: invalid token
```

```
>>> int('09')
```

#这样转换是可以的

```
9
```

```
>>> list(str([1, 2, 3, 4]))
```

#字符串中每个字符都变为列表中的元素

```
['[', '1', ',', ' ', '2', ',', ' ', '3', ',', ' ', '4', '']
```

```
>>> eval(str([1, 2, 3, 4]))
```

#字符串求值

```
[1, 2, 3, 4]
```

2.3.2 最值与求和

- `max()`、`min()`、`sum()`这三个内置函数分别用于计算列表、元组或其他包含有限个元素的可迭代对象中所有元素最大值、最小值以及所有元素之和。
- `sum()`默认（可以通过`start`参数来改变）支持包含数值型元素的序列或可迭代对象，`max()`和`min()`则要求序列或可迭代对象中的元素之间可比较大小。

```
>>> from random import randint
>>> a = [randint(1,100) for i in range(10)] #包含10个[1,100]之间随机数的列表
>>> print(max(a), min(a), sum(a))          #最大值、最小值、所有元素之和
>>> sum(a) / len(a)                        #平均值
```

2.3.2 最值与求和

- 函数max()和min()还支持default参数和key参数，其中default参数用来指定可迭代对象为空时默认返回的最大值或最小值，而key参数用来指定比较大小的依据或规则，可以是函数或lambda表达式。

```
>>> max(['2', '111'])           #不指定排序规则
'2'
>>> max(['2', '111'], key=len)   #返回最长的字符串
'111'
>>> print(max([], default=None)) #对空列表求最大值，返回空值None
None
```

2.3.3 基本输入输出

- `input()`和`print()`是Python的基本输入输出函数，前者用来接收用户的键盘输入，后者用来把数据以指定的格式输出到标准控制台或指定的文件对象。不论用户输入什么内容，`input()`一律返回字符串对待，必要的时候可以使用内置函数`int()`、`float()`或`eval()`对用户输入的内容进行类型转换。

2.3.3 基本输入输出

```
>>> x = input('Please input: ')
```

```
Please input: 345
```

```
>>> x
```

```
'345'
```

```
>>> type(x)
```

```
<class 'str'>
```

```
>>> int(x)
```

```
345
```

```
>>> eval(x)
```

```
345
```

```
>>> x = input('Please input: ')
```

```
Please input: [1, 2, 3]
```

```
>>> x
```

```
'[1, 2, 3]'
```

```
>>> type(x)
```

```
<class 'str'>
```

```
>>> eval(x)
```

```
[1, 2, 3]
```

#把用户的输入作为字符串对待

#转换为整数

#对字符串求值，或类型转换

2.3.3 基本输入输出

- 内置函数print()用于输出信息到标准控制台或指定文件，语法格式为：

`print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)`

- ✓ sep参数之前为需要输出的内容（可以有多个）；
- ✓ sep参数用于指定数据之间的分隔符，默认为空格；
- ✓ end参数用于指定输出完数据之后再输出什么字符；

```
>>> print(1, 3, 5, 7, sep='\t')
```

#修改默认分隔符

```
1      3      5      7
```

```
>>> for i in range(10):  
    print(i, end=' ')
```

#修改end参数，每个输出之后不换行

```
0 1 2 3 4 5 6 7 8 9
```

2.3.4 排序与逆序

- `sorted()`对列表、元组、字典、集合或其他可迭代对象进行排序并返回新列表，`reversed()`对可迭代对象（生成器对象和具有惰性求值特性的`zip`、`map`、`filter`、`enumerate`等类似对象除外）进行翻转（首尾交换）并返回可迭代的`reversed`对象。

```
>>> x = list(range(11))
>>> import random
>>> random.shuffle(x)                #打乱顺序
>>> x
[2, 4, 0, 6, 10, 7, 8, 3, 9, 1, 5]
>>> sorted(x)                        #以默认规则排序
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> sorted(x, key=lambda item:len(str(item)), reverse=True)
                                     #按转换成字符串以后的长度降序排列
[10, 2, 4, 0, 6, 7, 8, 3, 9, 1, 5]
>>> sorted(x, key=str)               #按转换成字符串以后的大小升序排列
[0, 1, 10, 2, 3, 4, 5, 6, 7, 8, 9]
```

2.3.4 排序与逆序

```
>>> x = ['aaaa', 'bc', 'd', 'b', 'ba']
>>> sorted(x, key=lambda item: (len(item), item))
#先按长度排序, 长度一样的正常排序
['b', 'd', 'ba', 'bc', 'aaaa']
>>> reversed(x)
#逆序, 返回reversed对象
<list_reverseiterator object at 0x000000003089E48>
>>> list(reversed(x))
#reversed对象是可选代的
['ba', 'b', 'd', 'bc', 'aaaa']
```

2.3.5 枚举与迭代

- `enumerate()`函数用来枚举可迭代对象中的元素，返回可迭代的`enumerate`对象，其中每个元素都是包含索引和值的元组。

```
>>> list(enumerate('abcd'))                                #枚举字符串中的元素
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd')]
>>> list(enumerate(['Python', 'Greate']))                  #枚举列表中的元素
[(0, 'Python'), (1, 'Greate')]
>>> list(enumerate({'a':97, 'b':98, 'c':99}.items()))        #枚举字典中的元素
[(0, ('c', 99)), (1, ('a', 97)), (2, ('b', 98))]
>>> for index, value in enumerate(range(10, 15)):           #枚举range对象中的元素
    print((index, value), end=' ')

(0, 10) (1, 11) (2, 12) (3, 13) (4, 14)
```

2.3.6 map()、reduce()、filter()

- 内置函数map()把一个函数func依次映射到序列或迭代器对象的每个元素上，并返回一个可迭代的map对象作为结果，map对象中每个元素是原序列中元素经过函数func处理后的结果。

```
>>> list(map(str, range(5))) #把列表中元素转换为字符串
['0', '1', '2', '3', '4']
>>> def add5(v): #单参数函数
    return v+5
>>> list(map(add5, range(10))) #把单参数函数映射到一个序列的所有元素
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
>>> def add(x, y): #可以接收2个参数的函数
    return x+y
>>> list(map(add, range(5), range(5,10)))
#把双参数函数映射到两个序列上
[5, 7, 9, 11, 13]
```

2.3.6 map()、reduce()、filter()

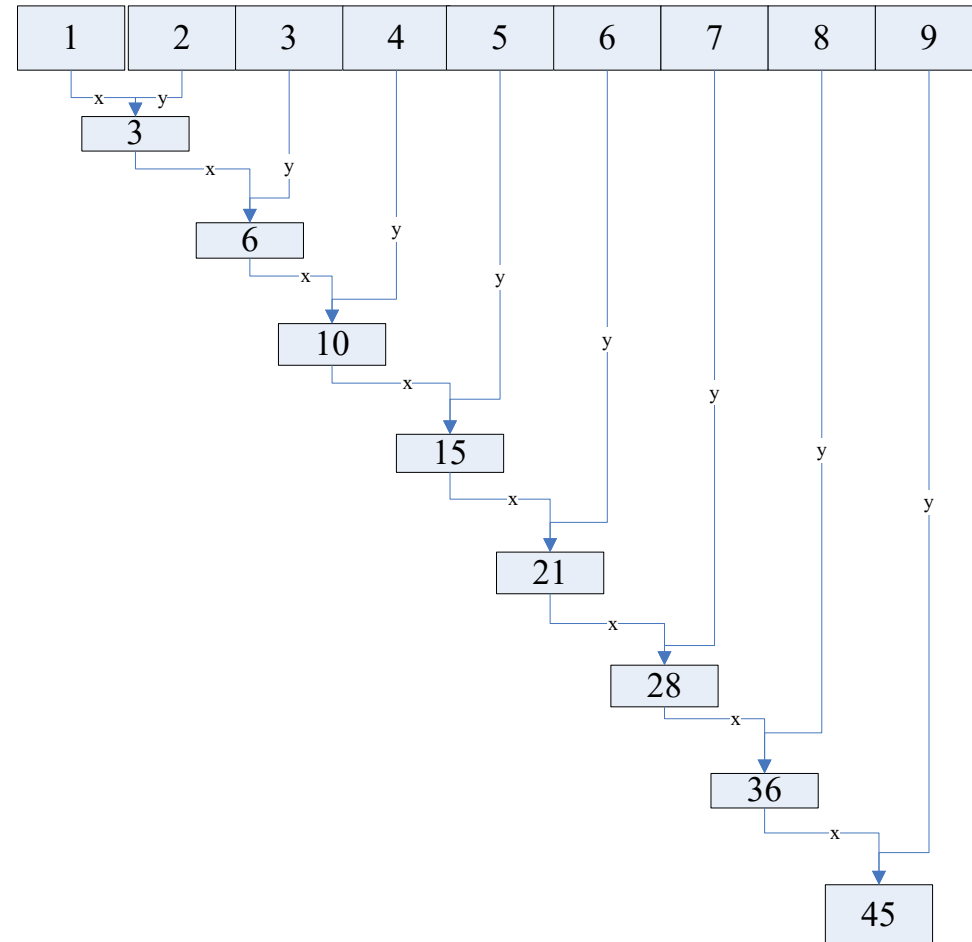
```
>>> import random
>>> x = random.randint(1, 1e30)      #生成指定范围内的随机整数
>>> x
839746558215897242220046223150
>>> list(map(int, str(x)))           #提取大整数每位上的数字
[8, 3, 9, 7, 4, 6, 5, 5, 8, 2, 1, 5, 8, 9, 7, 2, 4, 2, 2, 2, 0, 0, 4, 6,
2, 2, 3, 1, 5, 0]
```

2.3.6 map()、reduce()、filter()

- 标准库functools中的函数reduce()可以将一个接收2个参数的函数以迭代累积的方式从左到右依次作用到一个序列或迭代器对象的所有元素上，并且允许指定一个初始值。

```
>>> from functools import reduce
>>> seq = list(range(1, 10))
>>> reduce(lambda x, y: x+y, seq)
```

45



2.3.6 map()、reduce()、filter()

- 内置函数filter()将一个单参数函数作用到一个序列上，返回该序列中使得该函数返回值为True的那些元素组成的filter对象，如果指定函数为None，则返回序列中等价于True的元素。

```
>>> seq = ['foo', 'x41', '?!', '***']
```

```
>>> def func(x):
```

```
    return x.isalnum()
```

#测试是否为字母或数字

alphabet number

```
>>> filter(func, seq)
```

#返回filter对象

```
<filter object at 0x00000000305D898>
```

```
>>> list(filter(func, seq))
```

#把filter对象转换为列表

```
['foo', 'x41']
```

2.3.6 map()、reduce()、filter()

- 内置函数filter()将一个单参数函数作用到一个序列上，返回该序列中使得该函数返回值为True的那些元素组成的filter对象，如果指定函数为None，则返回序列中等价于True的元素。

```
>>> fo=[2, 5, 9, 20, 24, 27, 4, 18]
>>> print filter( lambda x: x%3 == 0, fo)
>>> [9, 24, 27, 18]
```

2.3.7 range()

- range()是Python开发中非常常用的一个内置函数，语法格式为range([start,] end [, step]), 有range(stop)、range(start, stop)和range(start, stop, step)三种用法。该函数返回具有惰性求值特点的range对象，其中包含左闭右开区间[start,end)内以step为步长的整数。参数start默认为0，step默认为1。

```
>>> range(5)                                #start默认为0， step默认为1
range(0, 5)
>>> list(_)
[0, 1, 2, 3, 4]
>>> list(range(1, 10, 2))                    #指定起始值和步长
[1, 3, 5, 7, 9]
>>> list(range(9, 0, -2))                    #步长为负数时， start应比end大
[9, 7, 5, 3, 1]
```

2.3.8 zip()

- zip()函数用来把多个可迭代对象中的元素压缩到一起，返回一个可迭代的zip对象，其中每个元素都是包含原来的多个可迭代对象对应位置上元素的元组，如同拉拉链一样。

```
>>> list(zip('abcd', [1, 2, 3]))           #压缩字符串和列表  
[('a', 1), ('b', 2), ('c', 3)]
```

```
>>> list(zip('123', 'abc', ',.!' ))       #压缩3个序列  
[('1', 'a', ','), ('2', 'b', '.'), ('3', 'c', '!')]
```

```
>>> x = zip('abcd', '1234')  
>>> list(x)  
[('a', '1'), ('b', '2'), ('c', '3'), ('d', '4')]
```



2.4 Python关键字简要说明

- Python关键字只允许用来表达特定的语义，**不允许**通过任何方式改变它们的含义，也**不能**用来做变量名、函数名或类名等标识符。
- 在Python开发环境中导入模块keyword之后，可以使用`print(keyword.kwlist)`查看所有关键字。

2.4 Python关键字简要说明

关键字	含义
False	常量，逻辑假
None	常量，空值
True	常量，逻辑真
and	逻辑与运算
as	在import或except语句中给对象起别名
assert	断言，用来确认某个条件必须满足，可用来帮助调试程序
break	用在循环中，提前结束break所在层次的循环
class	用来定义类
continue	用在循环中，提前结束本次循环
def	用来定义函数
del	用来删除对象或对象成员
elif	用在选择结构中，表示else if的意思
else	可以用在选择结构、循环结构和异常处理结构中
except	用在异常处理结构中，用来捕获特定类型的异常
finally	用在异常处理结构中，用来表示不论是否发生异常都会执行的代码
for	构造for循环，用来迭代序列或可迭代对象中的所有元素
from	明确指定从哪个模块中导入什么对象，例如from math import sin; 还可以与yield一起构成yield表达式

2.4 Python关键字简要说明

关键字	含义
global	定义或声明全局变量
if	用在选择结构中
import	用来导入模块或模块中的对象
in	成员测试
is	同一性测试
lambda	用来定义lambda表达式，类似于函数
nonlocal	用来声明nonlocal变量
not	逻辑非运算
or	逻辑或运算
pass	空语句，执行该语句时什么都不做，常用作占位符
raise	用来显式抛出异常
return	在函数中用来返回值，如果没有指定返回值，表示返回空值None
try	在异常处理结构中用来限定可能会引发异常的代码块
while	用来构造while循环结构，只要条件表达式等价于True就重复执行限定的代码块
with	上下文管理，具有自动管理资源的功能
yield	在生成器函数中用来返回值

课堂作业

1. 鸡兔同笼问题

输入头的个数和脚的个数，计算有多少只兔子多少只鸡

2. 分数评级，输入成绩分数，输出等级

“不及格”， “差”， “中”， “良”， “优”

60分以下 60-70 70-80 80-90 90-100

3. 输入x与n，输出 $\text{sum} = x + xx + xxx + xxxx + \dots + \text{xxxxxxx}$

n